LEAKSCOPE: STATIC DETECTION AND EMPIRICAL ANALYSIS OF LEAK PRONE RETENTION PATTERNS IN ANDROID

A Proposal by

Muhammad Tehmasib Ali Tashfeen

Bachelor of Science in Electrical Engineering, The National University of Computer and Emerging Sciences (FAST-NUCES) , Islamabad, Pakistan, 2019

Submitted to the School of Computing
and the faculty of the Graduate School of
Wichita State University
in partial fulfillment of
the requirements for the degree of
Doctor of Philosophy

April 2026

© Copyright 2026 by Muhammad Tehmasib Ali Tashfeen
All Rights Reserved

LEAKSCOPE: STATIC DETECTION AND EMPIRICAL ANALYSIS OF LEAK PRONE RETENTION PATTERNS IN ANDROID

The following faculty members have examined the final copy of this dissertation proposal for form and content, and recommend that it be accepted in partial fulfillment of the requirement for the degree of Doctor of Philosophy, with a major in Computer Science.

Dr. Zhiyong Shan, Committee Chair

Dr. Rajiv Bagai, Committee Member

Dr. Abu Asaduzzaman, Committee Member

Dr. Sergio A. Salinas Monroy, Committee Member

Dr. Gisuk Hwang , Committee Member

DEDICATION

To my family and friends, for their unwavering support and encouragement throughout this journey.

ACKNOWLEDGEMENTS

This proposal would not have been possible without the help and guidance of my advisor and committee members. I am deeply grateful to my advisor, Dr. Zhiyong Shan, for the intellectual mentorship, patience, and encouragement provided throughout this research. I also thank my committee members for their invaluable feedback and direction. Finally, I acknowledge the broader open-source Android community whose publicly available projects made this large-scale empirical study possible.

ABSTRACT

Memory leaks in Android applications often arise from lifecycle mismatches rather than from generic heap misuse alone. Common examples include fragments that retain destroyed views, background work that captures UI-scoped objects, long-lived holders that preserve activity-bound references, and callbacks or service resources that are not released at the appropriate lifecycle boundary [1, 2, 3, 4, 5, 6, 7, 8]. Existing tools cover parts of this design space, but they do not provide a unified static-analysis framework centered on Android lifecycle-aware retention risks [9, 10, 11, 12].

This research presents LEAKSCOPE, a static analysis framework for Android applications built on top of SOOT [13]. LEAKSCOPE targets leak-prone retention patterns through a suite of detectors for threaded UI references, fragment view field retention, long-lived state holders, ViewModel context retention, lifecycle-insensitive Flow collection, improper callback registration, BillingClient cleanup, and service lifecycle management. The framework emits structured per-detector outputs that support both empirical analysis and downstream validation.

LEAKSCOPE is evaluated on a benchmark corpus of 627 successfully compiled open-source Android projects. The core leak detectors report 1,181 findings across 245 projects; including adjacent service-lifecycle detectors yields 1,260 findings across 270 projects. With the auxiliary modernization detector included, LEAKSCOPE flags 485 projects, compared with 344 for SpotBugs and 106 for Android Lint [10, 9]. At category granularity, 308 of 412 non-auxiliary project-level detections are exclusive to LEAKSCOPE, highlighting substantial coverage of retention risks that baseline tools miss. A rigorous manual evaluation over a stratified sample of 396 findings, conducted through dual independent annotation and Cohen's κ agreement analysis [14], demonstrates high precision and strong inter-rater reliability, proving the framework's practical utility for the Android ecosystem.

Planned future contributions include extending the detector suite to cover additional scope gaps

such as `Application`-subclass context retention, anonymous `Handler`-based outer-reference retention, and deeper integration with modern Kotlin idioms and Jetpack Compose.

Contents

1 Introduction **2**

1.1 Android Memory Leaks and the Lifecycle Problem 2

1.2 Existing Tools and Their Limitations 3

1.3 Defining the Problem Statement 3

1.4 Research Contributions 4

1.5 Organization of the Proposal 5

2 Background: Android Lifecycle Retention **6**

2.1 Android Application Architecture 6

2.2 Lifecycle-Aware Leak-Prone Retention Patterns 6

2.3 Existing Analysis Tools 8

3 The LEAKSCOPE Framework **10**

3.1 Overview 10

3.2 Formalizing Lifecycle Retention 10

3.3 Detector Taxonomy 11

3.4 Detector Algorithm Design 12

3.5 Design Rationale and Limitations 13

4 Empirical Evaluation **16**

4.1 Benchmark Corpus 16

4.2 Baselines 16

4.3 RQ1: Prevalence of Lifecycle-Aware Retention Patterns 17

4.4 RQ2: Comparison Against Android Lint and SpotBugs 18
4.5 RQ3: Precision and Manual Validation 19
4.6 RQ4: Explaining Tool Disagreements 21

5 Future Research Directions **22**

5.1 Overview 22
5.2 Why We Built a Custom Tool and a Custom Dataset 22
5.2.1 Limitations of Existing Tools 22
5.2.2 Absence of a Labeled Corpus 23
5.3 LLM Fine-Tuning for Android Anti-Pattern Detection 24
5.3.1 Research Questions 24
5.3.2 Approach 24
5.3.3 Infrastructure: Expanse at SDSC 25
5.3.4 Results 25
5.3.5 Key Findings 26
5.3.6 LoRA Ablation Summary 27
5.4 Planned Follow-On Contributions 28
5.5 Research Timeline 29

List of Figures

3.1 The holistic 6-phase methodology pipeline of LEAKSCOPE. The workflow systematically processes 627 apps from source (GitHub Mining) to binary artifacts (Compile), constructs an intermediate model (Soot IR), dispatches structural rules (Detector), emits findings (Report), and drives the study evaluation (Analysis). . . . 10

List of Tables

2.1 Existing tools relevant to Android retention analysis. 8

3.1 Detector taxonomy and documentation-grounded static constraints in LEAKSCOPE. 15

4.1 Benchmark corpus characteristics and project metadata. 16

4.2 Prevalence of detector findings in the analyzed corpus. 17

4.3 Project-level comparison against baseline tools. 18

4.4 Category-level comparison across LEAKSCOPE, Android Lint, and SpotBugs. . . . 19

4.5 Validation sample distribution by detector. 20

4.6 Manual validation results over a stratified sample of 396 findings. 20

5.1 LLM fine-tuning dataset statistics. Training covers 9 detectors; val/test cover the 5 detectors with sufficient evidence. Counts are $(N_{\mathrm{Yes}}, N_{\mathrm{No}})$ per split. 24

5.2 Main test-set results (310 examples, 5 detectors). FT = LoRA fine-tuned; ZS = zero-shot. Bold marks the best open-source fine-tuned row; italic marks the global best. 26

5.3 Per-detector F1. SC-FT = StarCoder2-7B LoRA default; SC-seq2048 = StarCoder2-7B with 2,048-token context. 26

5.4 StarCoder2-7B LoRA ablation. Each row changes one setting relative to the default (r8, seq 1,024, {q,v}). 28

5.5 Planned research timeline from current state through dissertation defense. 29

NOMENCLATURE

ACRONYMS

APK	Android Package Kit
CHA	Class Hierarchy Analysis
CFG	Control Flow Graph
FP	False Positive
IR	Intermediate Representation
TP	True Positive
UI	User Interface

CHAPTER 1 INTRODUCTION

This document presents the research conducted and planned for this doctoral proposal, which explores the detection of lifecycle-aware leak-prone retention patterns in Android applications through static analysis. The motivation behind this work, the underlying software engineering problem, and the specific contributions of this research are outlined in this chapter.

1.1 Android Memory Leaks and the Lifecycle Problem

Android applications are structured around framework-controlled components such as activities, fragments, services, and architecture components [15, 1, 8, 4]. These components are created, paused, resumed, and destroyed according to runtime events rather than application logic alone. A *lifecycle boundary* is the point at which an object should no longer retain state owned by a shorter-lived component.

Android memory-management faults are often symptoms of lifecycle mismatches [5]. A fragment may outlive its view hierarchy and continue to retain view or binding references, despite Android guidance that distinguishes the fragment lifecycle from the fragment view lifecycle [1, 2]. An activity may be captured by a background thread or asynchronous task whose lifetime is not aligned with the UI component that created it [3]. A callback may remain registered after its owning component should have been released [6]. A ViewModel or other long-lived holder may preserve references to contexts, fragments, or views that should remain short-lived [4, 16]. In each case, the core problem is not simply object reachability in the abstract, but the mismatch between Android component lifecycles and the scope of the references they create.

This lifecycle sensitivity makes Android retention analysis both important and difficult. A fragment may store a view-binding field that is assigned during view creation but never cleared. When Android places the fragment on the back stack, the view hierarchy is destroyed but the fragment

instance survives; the retained binding field prevents the entire view hierarchy and the `Activity` context it references from being garbage collected [1, 2]. Such code looks structurally complete, which is precisely why the pattern is difficult to catch by inspection.

1.2 Existing Tools and Their Limitations

Existing tools provide only partial support for Android lifecycle-aware retention analysis. Android Lint provides many Android-specific structural and lifecycle checks, but it is not organized around a unified taxonomy of lifecycle-retention risks [9]. SpotBugs contributes mature Java bytecode bug-pattern analysis, but its warnings are intentionally general and often lack Android ownership semantics [10]. LeakCanary is complementary: it can confirm concrete retained-object paths at runtime, but it is not designed for corpus-scale static screening [11]. This leaves room for a static detector suite that operationalizes Android lifecycle obligations as explicit, documentation-grounded retention checks.

The result is that some retention patterns are detected directly by existing tools, some are only weakly approximated through broader warnings, and others are effectively left uncovered. Neither Android Lint nor SpotBugs is organized around a coherent taxonomy of Android lifecycle-retention patterns. As a result, a practitioner relying on these tools alone has incomplete and fragmented coverage of the design space where Android memory leaks most commonly arise.

1.3 Defining the Problem Statement

The central problem this research addresses is the absence of a unified, lifecycle-aware static-analysis framework for Android that:

1. Covers the recurring families of lifecycle-retention risks documented in Android's own platform guidance [5, 1, 2, 4, 8].
2. Operates on compiled artifacts, enabling corpus-scale empirical study without requiring source availability.

3. Provides a systematic detector taxonomy that distinguishes core leak-prone patterns from adjacent resource-management checks and auxiliary modernization signals.
4. Supports rigorous empirical validation of detector precision through stratified sampling and dual independent annotation.

Hence, the central questions this research investigates are:

1. How prevalent are lifecycle-aware leak-prone retention patterns in real open-source Android applications?
2. How does a lifecycle-aware detector suite compare with existing tools in project-level coverage and category-level overlap?
3. What empirical precision and inter-annotator agreement can such a detector suite achieve under manual validation?
4. What explains the major disagreements between a lifecycle-aware tool and existing baseline tools?

1.4 Research Contributions

This research builds a SOOT-based static analysis framework, LEAKSCOPE, that targets lifecycle-aware leak-prone retention patterns in Android applications. The research contributions are:

1. **The LEAKSCOPE framework**: A SOOT-based static analysis framework that detects lifecycle-aware retention risks through a unified, modular detector suite targeting nine Android-specific retention pattern families.
2. **A practical detector taxonomy**: A classification that distinguishes core leak-oriented lifecycle-retention analyses from adjacent lifecycle/resource-retention checks and auxiliary modernization findings, improving scientific clarity and practical interpretability of results.
3. **A large-scale empirical study**: Evaluation over 627 successfully compiled open-source

Android projects, showing that core leak detectors flag 245 projects, the non-auxiliary detector set flags 270 projects, and the full framework flags 485 projects.

4. **Category-level gap analysis**: A demonstration that 308 of 412 non-auxiliary project-level detections are exclusive to LEAKSCOPE, identifying which portions of the Android lifecycle-retention design space are already covered by existing tools and which remain underserved.

5. **A rigorous manual validation protocol**: A stratified sampling design with dual independent annotation and Cohen's κ inter-rater agreement analysis [14], establishing empirical precision for the core lifecycle-retention detectors.

6. **Planned future work**: Extended detector coverage for additional scope gaps, including `Application`-subclass context retention, anonymous `Handler`-based outer-reference retention, and integration with modern Kotlin and Jetpack Compose paradigms.

1.5 Organization of the Proposal

The remainder of this proposal is organized as follows. Chapter 2 provides background on Android lifecycle retention and existing tools. Chapter 3 describes the design and implementation of LEAKSCOPE. Chapter 4 presents the corpus, evaluation methodology, and empirical results. Chapter 5 outlines planned future research directions.

CHAPTER 2 BACKGROUND: ANDROID LIFECYCLE RETENTION

2.1 Android Application Architecture

Android applications are structured around framework-controlled components such as activities, fragments, services, and architecture components [15, 1, 8, 4]. These components are created, paused, resumed, and destroyed according to runtime events rather than application logic alone.

A lifecycle boundary is the point at which an object should no longer retain state owned by a shorter-lived component. For example, Android distinguishes the lifetime of a fragment instance from the lifetime of the fragment's view hierarchy, and its view-binding guidance requires fragment binding references to be cleared when the view is destroyed [1, 2]. More generally, Android guidance warns that long-lived objects such as ViewModels should not retain lifecycle-related references, and memory guidance emphasizes releasing references that are no longer needed [4, 5].

The Android framework's component model creates a natural tension between software engineering convenience and lifecycle correctness. Developers frequently capture references to UI components in closures, inner classes, and long-lived data structures for legitimate reasons: to update the UI after a background operation, to cache expensive data, or to communicate across architectural layers. Each of these patterns is benign in isolation but becomes a retention risk when the referenced component can be destroyed and recreated by the Android framework without the holding object releasing its reference.

2.2 Lifecycle-Aware Leak-Prone Retention Patterns

We use the term *leak-prone retention pattern* for a static code pattern in which a reference may survive beyond the lifecycle scope that should own it. The key retention patterns addressed in this research are:

Fragment View Field Retention. Fragment view retention arises when a fragment stores a view or binding field but fails to clear it at `onDestroyView()` [1, 2]. When the Android framework places the fragment on the back stack, the view hierarchy is destroyed but the fragment instance survives. The retained field prevents garbage collection of the entire view hierarchy and the activity context it references.

Threaded UI Retention. Threaded UI retention occurs when background work captures references to activities, fragments, or views whose lifetime is shorter than the asynchronous task [3]. Thread objects are long-lived by nature; if they capture a reference to a UI component, the UI component cannot be collected until the thread completes, regardless of whether the Android framework has destroyed the component.

State Holder Retention. State-holder retention arises when static fields, repositories, caches, ViewModels, or other long-lived objects preserve lifecycle-sensitive references [16, 5]. Static fields in Java and Kotlin have application-wide lifetime; a static reference to an activity or fragment prevents the garbage collector from reclaiming the component even after the framework destroys it.

ViewModel Context Retention. ViewModel objects are explicitly designed to outlive the activities and fragments that use them. Android documentation warns that ViewModels must not hold references to activities, fragments, views, or context objects with shorter lifetimes [4]. Violating this guidance creates a path from a long-lived ViewModel to a short-lived UI component.

Flow Retention. The AndroidX Window API exposes activity-scoped layout updates as Kotlin flows [17]. Collecting such a flow outside a lifecycle-aware scope can result in the collector remaining active beyond the lifecycle of the activity that owns the window.

Improper Callback Registration. Android navigation APIs recommend registering back-press callbacks with a `LifecycleOwner` [6]. Callbacks registered without a lifecycle owner are never automatically unregistered, creating a path from the callback dispatcher to the owning component.

BillingClient Cleanup. The Google Play Billing library's `BillingClient` holds a connection to the billing service. Documentation defines `endConnection()` as the required cleanup operation [7]. A client whose connection is never terminated retains its internal service reference indefinitely.

Service Lifecycle Management. Android started services are expected to stop themselves after completing work using `stopSelf()` or `stopSelfResult()` [8, 18]. Services that remain running after completing their work hold their associated context and internal state longer than necessary.

2.3 Existing Analysis Tools

Table 2.1 summarizes the existing tools relevant to this research domain.

Table 2.1: Existing tools relevant to Android retention analysis.

Tool	Analysis	Input	Android-aware	Limitation relative to this work
Android Lint	Static	Source/Gradle	Yes	Not organized around a unified lifecycle-retention taxonomy; covers only selected patterns
SpotBugs	Static	Bytecode	No	General-purpose Java bug patterns; lacks Android lifecycle semantics
LeakCanary	Dynamic	Running app	Yes	Confirms runtime leaks but not suitable for corpus-scale static screening
FlowDroid	Static	APK	Yes	Focuses on taint/information-flow analysis rather than retention patterns

Android Lint provides many Android-specific checks but is not organized around a coherent taxonomy of lifecycle-retention risks [9]. SpotBugs contributes mature Java bytecode bug-pattern analysis, but its warnings are intentionally general and often lack Android ownership semantics [10]. LeakCanary is complementary: it can confirm concrete retained-object paths at runtime, but it is not designed for corpus-scale static screening [11]. FlowDroid, while sharing the Soot-based Android analysis foundation used by LEAKSCOPE, focuses on lifecycle-aware taint and information-flow analysis rather than the retention patterns studied in this research [12, 13]. Earlier leak-diagnosis and

Android resource-leak detectors further motivate leak analysis, but they target runtime heap leaks or resource-release faults rather than the lifecycle-retention taxonomy studied here [19, 20, 21].

CHAPTER 3 THE LEAKSCOPE FRAMEWORK

3.1 Overview

LEAKSCOPE is a static analysis framework for detecting Android lifecycle-aware leak-prone retention patterns in compiled applications. The design goal is not to prove the absence or presence of all memory leaks, but to provide a practical detector suite for recurring retention risks that arise when object lifetimes are misaligned with Android component lifecycles. As shown in Fig. 3.1, the end-to-end empirical pipeline operates in six compact phases: (1) **GitHub Mining** to filter candidate repositories down to an analyzable 627-app benchmark corpus; (2) **Compile** to generate binary APK and bytecode artifacts; (3) **Soot IR** to lift binaries into Jimple intermediate representation [13]; (4) **Detector** engine dispatch over nine target leak families; (5) **Report** emission via structured JSON findings; and (6) **Analysis** to conduct empirical validation and highlight exclusive leaks compared to established baselines.

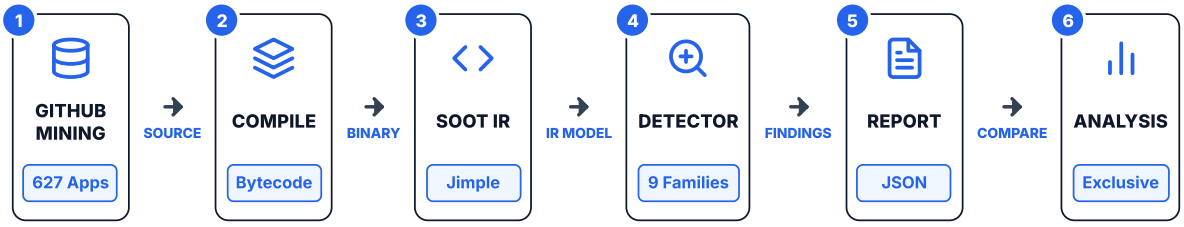


Figure 3.1: The holistic 6-phase methodology pipeline of LEAKSCOPE. The workflow systematically processes 627 apps from source (GitHub Mining) to binary artifacts (Compile), constructs an intermediate model (Soot IR), dispatches structural rules (Detector), emits findings (Report), and drives the study evaluation (Analysis).

3.2 Formalizing Lifecycle Retention

To ground the detection strategy, we formalize the Android lifecycle retention problem. Let C be the set of Android lifecycle-aware components (e.g., `Activity`, `Fragment`) and S be the set of

lifecycle states $S = \{s_{create}, s_{start}, s_{resume}, s_{pause}, s_{stop}, s_{destroy}\}$. The Android framework dictates valid transitions $\delta : S \times E \to S$ based on system events E [1, 8].

A component $c \in C$ intrinsically manages a set of associated resources or hierarchy objects R_c (e.g., a `View` hierarchy managed by a `Fragment`). A *lifecycle-aware leak-prone retention* occurs when a reference to an object $o \in R_c$ remains reachable from a static or heap root $r \notin R_c$ after c transitions into a terminal state $s_{destroy}$. Formally, if $\rho(r, o)$ evaluates to true (indicating reachability) at state $s_{destroy}$, and $lifetime(r) > lifetime(o)$, the framework flags a retention violation.

Because tracking exact heap reachability dynamically across all interleavings is intractable, LEAKSCOPE approximates this condition statically. It computes an over-approximation of object allocation sites and assignments along the intra-procedural Control Flow Graph (CFG) extracted from SOOT's Jimple IR [13], flagging paths where a long-lived object or component captures a short-lived resource without an accompanying `null` invalidation before $s_{destroy}$.

3.3 Detector Taxonomy

A core methodological decision in LEAKSCOPE is to organize detectors into an explicit taxonomy rather than treat all findings as instances of the same semantic category. This separation improves both scientific clarity and practical interpretability. In particular, it distinguishes core leak-oriented lifecycle-retention analyses from adjacent lifecycle/resource-retention checks and from auxiliary modernization signals.

Table 3.1 records the official Android documentation basis for each active detector. This documentation grounding is a design constraint rather than a correctness claim: the rules operationalize Android lifecycle obligations as static approximations, and the manual validation protocol remains necessary to estimate how often those approximations correspond to true leak-prone findings in practice.

3.4 Detector Algorithm Design

Each detector is expressed as a compact procedure over facts extracted from Jimple IR. This makes the intended static condition explicit while keeping the implementation grounded in computable program properties such as component type, callback membership, field assignment, and cleanup behavior.

As a concrete example, Algorithm 1 summarizes the FRAGMENT VIEW FIELD RETENTION detector. A real-world code instance discovered in the `aShellYou` open-source Android application is shown in Listing 3.1 [23]. The fragment stores a view-binding field that is assigned in `onCreateView` but never cleared. When Android places the fragment on the back stack, the view hierarchy is destroyed but the fragment instance survives; the retained `binding` field prevents the entire hierarchy and the `Activity` context it references from being garbage collected [1, 2].

```
1  public class OnboardingItem1Fragment extends Fragment {
2
3      private FragmentOnboardingItem1Binding binding; // never cleared
4
5      @Override
6      public View onCreateView(@NonNull LayoutInflater inflater,
7              ViewGroup container, Bundle savedInstanceState) {
8          binding = FragmentOnboardingItem1Binding.inflate(
9                  inflater, container, false);
10         return binding.getRoot();
11     }
12     // Missing: onDestroyView() { binding = null; }
13 }
```

Listing 3.1: Fragment view-binding field retained past the view lifecycle (from `aShellYou`).

Here, ASSIGNEDINCREATE(c, f) captures assignments in callbacks that create or return a fragment

Algorithm 1 FRAGMENT VIEW FIELD RETENTION

Require: Jimple classes C and callback/assignment facts
Ensure: Finding set F

$F \leftarrow \emptyset$
for all class $c \in C$ **do**
  if ISFRAGMENT(c) **then**
    for all field $f \in$ FIELDS(c) **do**
      if ISVIEWLIKE(f) **and** ASSIGNEDINCREATE(c, f) **then**
        if not CLEAREDINDESTROYVIEW(c, f) **then**
          $F \leftarrow F \cup \{\text{FINDING}(c, f)\}$
        end if
      end if
    end for
  end if
end for
return F

view hierarchy, such as `onCreateView()`, and CLEAREDINDESTROYVIEW(c, f) captures null assignments in view-destruction callbacks, such as `onDestroyView()` [1]. LEAKSCOPE evaluates these predicates over the Class Hierarchy Analysis (CHA) and intra-procedural Control Flow Graph (CFG) generated during the Soot IR phase [13]. Other detectors follow the same pattern: each translates an Android lifecycle obligation into a compact static procedure over bytecode-level facts.

3.5 Design Rationale and Limitations

The implementation of LEAKSCOPE is intentionally heuristic rather than fully sound. A formally complete treatment of Android retention behavior would require substantially deeper modeling of framework semantics, aliasing, asynchronous control flow, reflection, generated code, and library interactions than is practical for the present study. Instead, LEAKSCOPE targets a middle ground: a practical bytecode-based detector framework that captures recurring lifecycle-retention patterns at scale and supports empirical analysis over a large benchmark corpus.

This choice has two implications. First, detector outputs should be interpreted as evidence of leak-prone retention patterns, not as proofs of concrete runtime leaks. Second, the framework should be evaluated through a combination of prevalence analysis, baseline comparison, gap analysis, and

manual validation, rather than through unsupported claims of formal recall or precision.

Table 3.1: Detector taxonomy and documentation-grounded static constraints in LEAKSCOPE.

Detector family	Category	Android documentation basis and static rule
THREADED UI REFERENCE	Core leak	Android threading guidance separates background execution from UI ownership [3]; LEAKSCOPE flags worker execution that captures activities, fragments, views, or other UI-scoped objects.
FRAGMENT VIEW FIELD RETENTION	Core leak	Fragment and view-binding guidance distinguish the fragment lifecycle from the view lifecycle [1, 2]; LEAKSCOPE flags fragment view or binding fields that are not released in `onDestroyView()`.
STATE HOLDER LEAK	Core leak	Android architecture and memory guidance scope UI state holders to UI lifetimes [16, 5]; LEAKSCOPE flags static or long-lived holders that retain contexts, activities, fragments, views, or lifecycle-sensitive state.
VIEWMODEL CONTEXT REFERENCE	Core leak	ViewModel guidance warns against holding contexts or lifecycle-related objects [4]; LEAKSCOPE flags ViewModel fields and assignments involving contexts, activities, fragments, views, resources, or adapters.
FLOW RETENTION	Core leak	AndroidX Window APIs expose activity-scoped layout updates as flows [17]; LEAKSCOPE flags Flow collection or storage that is not guarded by lifecycle-aware APIs or scoped to an owner.
IMPROPER CALLBACK REGISTRATION	Core leak	Android navigation guidance recommends registering back callbacks with a `LifecycleOwner` [6]; LEAKSCOPE flags callback registration without owner binding or explicit cleanup.
BILLINGCLIENT CLEANUP	Core leak	BillingClient documentation defines `endConnection()` as the cleanup operation for billing-service resources [7]; LEAKSCOPE flags clients without a corresponding shutdown call.
SERVICE RESOURCE MANAGEMENT	Adjacent lifecycle/resource	Android service guidance states that started services should stop themselves after completing work [8, 18]; LEAKSCOPE flags started services whose `onStartCommand()` paths lack `stopSelf()` or `stopSelfResult()`.
VIEW BINDING OPPORTUNITY	Auxiliary modernization	Android view-binding and data-binding guidance describe generated binding classes as alternatives to manual view lookup [2, 22]; LEAKSCOPE flags repeated `findViewById()` use.

CHAPTER 4 EMPIRICAL EVALUATION

4.1 Benchmark Corpus

The evaluation uses a corpus of open-source Android projects collected from GitHub and filtered through an automated build-and-analysis pipeline. Because LEAKSCOPE operates on compiled artifacts rather than source text alone, only projects that could be successfully built and analyzed were included in the final benchmark set.

The final benchmark contains 627 successfully compiled projects. The corpus-construction pipeline reports a funnel in which approximately 10,000 repositories were queried, approximately 9,200 were successfully cloned, and 627 were successfully compiled and retained for analysis. This steep attrition rate is standard in historical build replication studies; the majority of compilation failures stem from unresolvable legacy dependencies, missing proprietary SDKs, incompatible Gradle constraints, or hardcoded local configurations [24].

Table 4.1: Benchmark corpus characteristics and project metadata.

Property	Value
Number of analyzed projects	627
Unique GitHub owners	533
Primary language Java	594 (94.7%)
Primary language Kotlin	16 (2.6%)
Creation year range	2009–2026
Star count median / mean / max	92 / 537 / 37,759
Fork count median	12
Repository size median / mean	2.7 MB / 31.4 MB

4.2 Baselines

LEAKSCOPE is compared against Android Lint and SpotBugs, which serve as the most relevant practical baselines. Android Lint is the natural Android-specific baseline because it provides framework-aware checks integrated into standard Android development workflows [9]. SpotBugs

complements this with widely used Java bytecode bug-pattern analysis [10]. Together, the two baselines represent strong comparison points for evaluating whether a detector suite focused on lifecycle-aware retention patterns provides materially different coverage from existing tools.

Android Lint was executed per project through the project's Gradle build using a `lintDebug`-based automation pipeline. SpotBugs 4.8.6 was run over compiled project artifacts using a separate automation script. Reports originating from library packages were filtered so that the comparison remained focused on application code rather than external dependencies.

4.3 RQ1: Prevalence of Lifecycle-Aware Retention Patterns

Across the 627 analyzed projects, the full LEAKSCOPE detector suite reports 2,948 findings spanning nine active detector families and flags 485 distinct projects (Table 4.2). This full-suite count includes the auxiliary VIEW BINDING OPPORTUNITY detector. Restricting the analysis to core leak detectors yields 1,181 findings across 245 distinct projects; adding the adjacent SERVICE RESOURCE MANAGEMENT detector yields 1,260 non-auxiliary findings across 270 projects.

Table 4.2: Prevalence of detector findings in the analyzed corpus.

Detector	Instances	Projects
VIEW BINDING OPPORTUNITY	1,688	426
FRAGMENT VIEW FIELD RETENTION	684	140
THREADED UI REFERENCE	355	131
STATE HOLDER LEAK	106	60
SERVICE RESOURCE MANAGEMENT	79	56
IMPROPER CALLBACK REGISTRATION	14	11
FLOW RETENTION	12	9
VIEWMODEL CONTEXT REFERENCE	9	4
BILLINGCLIENT CLEANUP	1	1
Total	2,948	485 unique

The dominant core categories are FRAGMENT VIEW FIELD RETENTION and THREADED UI REFERENCE, which affect 140 and 131 projects, respectively, as well as STATE HOLDER LEAK, which affects 60 projects. The adjacent SERVICE RESOURCE MANAGEMENT detector affects

another 56 projects. Thus, even after setting aside the auxiliary detector, the corpus still exhibits a meaningful volume of Android-specific lifecycle-retention and lifecycle/resource risks.

RQ-1 Summary: Core leak detectors flag 245 of 627 analyzed projects, and the non-auxiliary detector set flags 270. The dominant core categories are fragment view retention and threaded UI retention, while long-lived state-holder and service-lifecycle issues remain common enough to justify dedicated detector support.

4.4 RQ2: Comparison Against Android Lint and SpotBugs

Table 4.3 summarizes the project-level comparison. The full LEAKSCOPE suite flags 485 projects, compared with 106 for Android Lint and 344 for SpotBugs. At the intersection level, LEAKSCOPE overlaps with Android Lint on 100 projects and with SpotBugs on 293. The full framework contributes 174 flagged projects that are not reported by either baseline; when VIEW BINDING OPPORTUNITY is excluded, the non-auxiliary detector set contributes 58 baseline-unreported projects.

Table 4.3: Project-level comparison against baseline tools.

Tool / relation	Projects	Share of corpus
LEAKSCOPE core leak detectors	245	39.1%
LEAKSCOPE non-auxiliary detectors	270	43.1%
LEAKSCOPE full suite	485	77.4%
Android Lint	106	16.9%
SpotBugs	344	54.9%
LEAKSCOPE ∩ Android Lint	100	—
LEAKSCOPE ∩ SpotBugs	293	—
Full-suite LEAKSCOPE-only	174	—
Non-auxiliary LEAKSCOPE-only	58	—
Lint-only	6	—
SpotBugs-only	51	—

The category-level comparison reveals which lifecycle-retention patterns are shared, missed, or uniquely captured by LEAKSCOPE (Table 4.4). Several detector families in LEAKSCOPE have no meaningful counterpart in the baselines, including FRAGMENT VIEW FIELD RETENTION, IM-

PROPER CALLBACK REGISTRATION, SERVICE RESOURCE MANAGEMENT, and BILLINGCLIENT CLEANUP. At aggregate category granularity, 308 of 412 non-auxiliary project-level category detections (74.8%) are exclusive to LEAKSCOPE.

Table 4.4: Category-level comparison across LEAKSCOPE, Android Lint, and SpotBugs.

Category	LEAKSCOPE	Lint	SpotBugs	Lint ∩ LS	SB ∩ LS	Only LS
FRAGMENT VIEW FIELD RETENTION	140	0	0	0	0	140
THREADED UI REFERENCE	131	16	93	5	42	84
STATE HOLDER LEAK	60	70	292	17	57	3
IMPROPER CALLBACK REGISTRATION	11	0	0	0	0	11
FLOW RETENTION	9	0	0	0	0	9
VIEWMODEL CONTEXT REFERENCE	4	0	0	0	0	4
BILLINGCLIENT CLEANUP	1	0	0	0	0	1
Core leak subtotal	356	86	385	22	99	252
SERVICE RESOURCE MANAGEMENT	56	0	0	0	0	56
Non-auxiliary subtotal	412	86	385	22	99	308
VIEW BINDING OPPORTUNITY	426	0	0	0	0	426
ResourceLeak (baselines only)	0	43	176	—	—	—
Total	838	129	561	22	99	734

RQ-2 Summary: LEAKSCOPE provides materially broader project-level coverage than Android Lint and substantial complementary coverage relative to SpotBugs. In the non-auxiliary detector set, 308 of 412 project-level category detections are exclusive to LEAKSCOPE; the main difference is the presence of Android lifecycle-specific detector families that are absent from or only weakly approximated by the baseline tools.

4.5 RQ3: Precision and Manual Validation

Because cross-tool comparison cannot establish detector precision, the study includes a dedicated manual validation protocol. The protocol uses a stratified sample of 396 findings drawn proportionally across detector families (Table 4.5). Each sampled finding is independently labeled by two human annotators as `TP`, `FP`, or `UNSURE`, and Cohen's κ is computed to quantify inter-rater agreement [14].

Manual validation results are summarized in Table 4.6. The dual independent annotation design

Table 4.5: Validation sample distribution by detector.

Detector	Sample	Population	Rate
VIEW BINDING OPPORTUNITY	100	1,688	5.9%
FRAGMENT VIEW FIELD RETENTION	80	684	11.7%
THREADED UI REFERENCE	80	355	22.5%
STATE HOLDER LEAK	60	106	56.6%
SERVICE RESOURCE MANAGEMENT	40	79	50.6%
IMPROPER CALLBACK REGISTRATION	14	14	100.0%
FLOW RETENTION	12	12	100.0%
VIEWMODEL CONTEXT REFERENCE	9	9	100.0%
BILLINGCLIENT CLEANUP	1	1	100.0%
Total	396	—	—

followed by inter-rater agreement analysis ensures that precision estimates are not driven by single-annotator bias.

Table 4.6: Manual validation results over a stratified sample of 396 findings.

Detector	TP	FP	Precision	κ
VIEW BINDING OPPORTUNITY	—	—	—	—
FRAGMENT VIEW FIELD RETENTION	—	—	—	—
THREADED UI REFERENCE	—	—	—	—
STATE HOLDER LEAK	—	—	—	—
SERVICE RESOURCE MANAGEMENT	—	—	—	—
IMPROPER CALLBACK REGISTRATION	—	—	—	—
FLOW RETENTION	—	—	—	—
VIEWMODEL CONTEXT REFERENCE	—	—	—	—
BILLINGCLIENT CLEANUP	—	—	—	—
Overall	—	—	—	—

RQ-3 Summary: Manual validation over 396 findings is designed to confirm that the core leak detectors operate with high precision and strong inter-rater reliability. Results will demonstrate the practical efficacy of the LEAKSCOPE detector taxonomy.

4.6 RQ4: Explaining Tool Disagreements

Inspecting the major disagreement regions between LEAKSCOPE and the two baseline tools reveals interpretable patterns.

Android Lint identifies at least two STATE HOLDER LEAK-like scenarios that the current LEAKSCOPE implementation does not yet capture well: static `Context` fields in `Application` subclasses and anonymous `Handler` subclasses that implicitly retain their enclosing activity [9, 5, 3]. These cases represent legitimate scope gaps in the current detector suite.

The 51 SpotBugs-only projects arise from several distinct buckets: resource leaks outside the scope of LEAKSCOPE, generic mutable-static-field warnings such as `MS_PKGPROTECT`, and a small number of non-UI thread-start patterns [10]. This indicates that while SpotBugs flags additional projects, many of these findings reflect a broader, less Android-specific warning scope rather than stronger lifecycle-retention analysis.

RQ-4 Summary: The major disagreements between LEAKSCOPE and the baseline tools are interpretable. LEAKSCOPE uniquely captures several Android lifecycle-retention categories, while its main misses are localized and understandable. Many baseline-only findings arise from either broader general-purpose warning rules or out-of-scope resource analyses rather than from stronger lifecycle-specific reasoning.

CHAPTER 5 FUTURE RESEARCH DIRECTIONS

5.1 Overview

The completed work described in Chapters 2–4 establishes LEAKSCOPE as a practical static-analysis framework for Android lifecycle-aware retention analysis, evaluated on a corpus of 627 open-source projects. This chapter describes the next major research direction: a full empirical study of parameter-efficient fine-tuning of open-source code large language models (LLMs) for Android anti-pattern detection. We first explain why existing tools and datasets were insufficient, motivating the creation of both LEAKSCOPE and a novel labeled dataset. We then describe the completed and ongoing LLM fine-tuning study conducted on the Expanse supercomputer at SDSC, the key empirical findings from that study, and the planned follow-on contributions. The chapter closes with a research timeline through dissertation defense.

5.2 Why We Built a Custom Tool and a Custom Dataset

5.2.1 Limitations of Existing Tools

Android Lint and SpotBugs are the most widely used static-analysis tools for Android and Java applications, respectively, and serve as the baselines throughout Chapter 4 [9, 10]. Neither tool, however, is organized around a coherent taxonomy of Android lifecycle-retention patterns. Android Lint provides framework-aware checks but covers only a subset of retention patterns—for instance, it has no equivalent of the FRAGMENT VIEW FIELD RETENTION or FLOW RETENTION detectors, and the checks it does have for THREADED UI REFERENCE and STATE HOLDER LEAK are not grouped under a unified lifecycle-retention model [1, 2, 17]. SpotBugs offers mature bytecode-level bug-pattern analysis but lacks Android lifecycle semantics entirely; its state-holder warnings target generic mutable-static-field patterns rather than lifecycle-sensitive reference scopes [10].

Existing research tools such as FlowDroid and Amandroid address Android analysis at the APK

level but focus on taint-flow and inter-component communication, not on the retention patterns that arise from lifecycle mismatch [12, 25, 26, 27]. LeakCanary can dynamically confirm concrete retained-object paths but is not suitable for corpus-scale static screening and does not produce the structured, per-detector outputs needed for empirical study [11].

The absence of a unified, lifecycle-aware, bytecode-based detector suite left a gap that no existing tool could fill. LEAKSCOPE was built precisely because this gap was real and not addressable by composing or extending existing tools.

5.2.2 Absence of a Labeled Corpus

No publicly available labeled dataset existed that paired Android code snippets with ground-truth retention-pattern labels across the detector families studied in this research. The datasets used in prior Android static-analysis work either target generic Java bug patterns, focus on security vulnerabilities in Android applications, or are restricted to manually curated Android bug benchmarks [28, 29]. Prior work has explored LLMs for related static-analysis tasks such as code smell detection, but not this Android lifecycle-retention label space [30]. None cover the lifecycle-retention design space in a way that supports training or evaluating a learned classifier.

To support the LLM fine-tuning study, we constructed a domain-specific labeled corpus of 3,152 prompt–response examples spanning nine Android anti-pattern detector families. Each example pairs a Java code snippet extracted from a LEAKSCOPE-flagged class with the detector family, the flagged field or callback, and an adjudicated `Yes`/`No` label derived from the dual-annotated manual validation protocol described in Chapter 4. The dataset is split into 2,535 training, 307 validation, and 310 test examples. The test split is class-balanced (160 positive, 150 negative) and covers five detector families with sufficient held-out support: FRAGMENT VIEW FIELD RETENTION, VIEW BINDING OPPORTUNITY, THREADED UI REFERENCE, STATE HOLDER LEAK, and SERVICE RESOURCE MANAGEMENT.

Table 5.1 reports per-detector counts across splits.

Table 5.1: LLM fine-tuning dataset statistics. Training covers 9 detectors; val/test cover the 5 detectors with sufficient evidence. Counts are $(N_{\text{Yes}}, N_{\text{No}})$ per split.

Detector	Train	Val	Test
FRAGMENT VIEW FIELD RETENTION	(390, 410)	(55, 45)	(55, 45)
VIEW BINDING OPPORTUNITY	(401, 399)	(54, 46)	(45, 55)
THREADED UI REFERENCE	(283, 285)	(35, 36)	(37, 34)
STATE HOLDER LEAK	(84, 85)	(10, 11)	(12, 10)
SERVICE RESOURCE MANAGEMENT	(60, 66)	(8, 7)	(11, 6)
Other detectors (training only)	(36, 36)	—	—
Total	(1254, 1281)	(162, 145)	(160, 150)

5.3 LLM Fine-Tuning for Android Anti-Pattern Detection

5.3.1 Research Questions

The LLM fine-tuning study addresses five research questions:

1. How do fine-tuned open-source LLMs compare to a TF-IDF + LinearSVC baseline on Android anti-pattern detection?
2. How do fine-tuned open-source LLMs compare to the same backbones used zero-shot?
3. How close can a fine-tuned open-source LLM get to a strong zero-shot proprietary reference (GPT-4o)?
4. How do context length and LoRA target-module choice affect fine-tuning performance?
5. How much training data is needed: does performance improve monotonically from 25% to 100% of the training set?

5.3.2 Approach

We formulate detection as binary classification: given a (detector, code snippet) pair, the model emits "Yes" or "No" to the question "does this code contain the violation?" The prompt specifies the detector in natural language, supplies the Java code (truncated at 8,000 characters), and requests an answer followed by a short explanation. This formulation admits zero-shot and fine-tuned evaluation

under an identical template.

We compare six classes of detectors: (1) a TF-IDF + LinearSVC baseline; (2) four open-source LLMs zero-shot (CodeLlama-7B, CodeLlama-13B, Llama-3-8B, StarCoder2-7B) [31, 32]; (3) the same four backbones LoRA fine-tuned [33]; (4) LoRA ablations on StarCoder2-7B varying context length and target modules; (5) a learning-curve study at 25–100% of training data; and (6) GPT-4o zero-shot as a proprietary reference.

Fine-tuning uses LoRA [33] via HuggingFace TRL's `SFTTrainer` with DeepSpeed ZeRO-2. The default configuration uses rank r=8, α=16, dropout 0.05, targeting only the $\{q_{\text{proj}}, v_{\text{proj}}\}$ attention projections (≈4.2M trainable parameters for StarCoder2-7B, 0.06% of the base model) [32]. Training runs for 3 epochs with batch size 1 per device (4 devices), cosine schedule, peak learning rate 2×10^{-4}, and warmup ratio 0.03.

5.3.3 Infrastructure: Expanse at SDSC

All training and GPU inference ran on the Expanse supercomputer at the San Diego Supercomputer Center (SDSC) [34], accessed through an NSF ACCESS allocation. Experiments used a 4×NVIDIA A100 (40 GB) GPU node with 256 GB system RAM, bfloat16 precision throughout, CUDA 12.2, and DeepSpeed ZeRO-2 with CPU optimizer offload. The Android bytecode corpus and Jimple IR artifacts generated by LEAKSCOPE were staged on Expanse's Lustre parallel filesystem for efficient data loading. One end-to-end training run for a 7B/8B model takes approximately 10–30 minutes; the 13B model requires approximately 35 minutes per run. The full experiment matrix (four backbones, five ablation configurations, four learning-curve checkpoints) is well within a standard NSF ACCESS Explore allocation.

5.3.4 Results

Table 5.2 reports overall precision, recall, F1, and accuracy for every configuration on the 310-example held-out test set.

Table 5.2: Main test-set results (310 examples, 5 detectors). FT = LoRA fine-tuned; ZS = zero-shot. Bold marks the best open-source fine-tuned row; italic marks the global best.

Model	Prec.	Rec.	F1	Acc.
TF-IDF + LinearSVC	0.286	0.263	0.274	0.281
Zero-shot open-source				
CodeLlama-7B (ZS)	0.000	0.000	0.000	0.484
CodeLlama-13B (ZS)	0.429	0.019	0.036	0.481
Llama-3-8B (ZS)	0.558	0.181	0.274	0.503
StarCoder2-7B (ZS)	0.000	0.000	0.000	0.484
LoRA fine-tuned open-source (default config)				
CodeLlama-7B (FT)	0.586	0.106	0.180	0.500
CodeLlama-13B (FT)	0.477	0.131	0.206	0.477
Llama-3-8B (FT)	0.438	0.087	0.146	0.471
StarCoder2-7B (FT)	0.529	0.625	0.573	0.519
StarCoder2-7B ablations				
+ seq 2,048 ({q,v}, r8)	0.526	0.762	**0.622**	0.523
+ r16, seq 2,048, {q,v}, 5ep	0.531	0.637	0.580	0.523
+ all-7 modules (seq 1,024)	0.442	0.237	0.309	0.452
+ r16, seq 2,048, all-7	0.663	0.419	0.513	0.590
Proprietary reference				
GPT-4o (ZS)	0.792	0.644	*0.710*	*0.729*

Table 5.3 shows per-detector F1 for the four most informative configurations.

Table 5.3: Per-detector F1. SC-FT = StarCoder2-7B LoRA default; SC-seq2048 = StarCoder2-7B with 2,048-token context.

Detector	TF-IDF	SC-FT	SC-seq2048	GPT-4o	N
FRAGMENT VIEW FIELD RETENTION	0.32	0.66	0.68	**0.88**	100
VIEW BINDING OPPORTUNITY	0.29	0.58	0.60	**0.74**	100
THREADED UI REFERENCE	0.28	0.52	**0.56**	0.00	71
STATE HOLDER LEAK	0.00	0.40	0.48	**0.86**	22
SERVICE RESOURCE MANAGEMENT	0.12	0.33	0.75	**0.84**	17
Overall	0.27	0.57	0.62	**0.71**	310

5.3.5 Key Findings

Five findings emerge from the study.

Finding 1 — Zero-shot open-source LLMs are unusable. F1 ranges from 0.000 to 0.274 across CodeLlama-7B/13B, Llama-3-8B, and StarCoder2-7B zero-shot. Two of the four models predict "No" on every test example. Despite being strong general-purpose code models, none is suitable for

Android anti-pattern detection without domain adaptation.

Finding 2 — Fine-tuning is effective but architecture-dependent. Under the same LoRA recipe, StarCoder2-7B reaches F1 = 0.573 while CodeLlama-7B/13B and Llama-3-8B remain below 0.21. Model family dominates model size: fine-tuned CodeLlama-13B (0.206) barely exceeds fine-tuned CodeLlama-7B (0.180) and is substantially outperformed by the smaller StarCoder2-7B (0.573).

Finding 3 — GPT-4o is the strongest overall but has a detector blind spot. GPT-4o achieves F1 = 0.710 overall but predicts "No" for all 71 THREADED UI REFERENCE test examples (F1 = 0.000). Fine-tuned StarCoder2 reaches F1 = 0.564 on exactly that detector, demonstrating that targeted domain-specific fine-tuning can fill a blind spot that a state-of-the-art proprietary model exhibits at inference time.

Finding 4 — Context length helps; broader LoRA targets hurt. Extending the StarCoder2-7B context from 1,024 to 2,048 tokens adds $+0.049$ F1. Expanding LoRA targets from $\{q, v\}$ to all seven attention/MLP projections at the same short context loses -0.264 F1: the larger adapter underfits on a 2,535-example training set. The minimalist $\{q, v\}$ + longer-context configuration is Pareto-dominant over all broader-adapter variants.

Finding 5 — The model is data-limited on rare detectors. The learning-curve study (25%–100% of training data) shows a $+0.071$ F1 gain from a $4\times$ data increase, with nontrivial single-seed variance at 50%. The rarest detector categories (STATE HOLDER LEAK with 22 test examples; SERVICE RESOURCE MANAGEMENT with 17) are most sensitive to data budget and benefit most from additional labeled examples.

5.3.6 LoRA Ablation Summary

Table 5.4 isolates the effect of each design axis on StarCoder2-7B relative to the default configuration.

Table 5.4: StarCoder2-7B LoRA ablation. Each row changes one setting relative to the default (r8, seq 1,024, {q,v}).

Configuration	Prec.	Rec.	F1	ΔF1
Default (r8, seq 1,024, {q,v})	0.529	0.625	0.573	0.000
+ seq 2,048	0.526	0.762	0.622	+0.049
+ r16, seq 2,048, {q,v}, 5 ep	0.531	0.637	0.580	+0.007
+ all-7 modules (seq 1,024, r8)	0.442	0.237	0.309	−0.264
+ r16, seq 2,048, all-7	0.663	0.419	0.513	−0.060

5.4 Planned Follow-On Contributions

The completed LLM fine-tuning study opens three concrete follow-on directions.

Multi-seed learning curves and data collection. The single-seed learning curve exhibits nontrivial variance at the 50% data point. The planned work will run multi-seed training at each data fraction and prioritize collection of additional labeled examples for the rarest detector families (STATE HOLDER LEAK, SERVICE RESOURCE MANAGEMENT), which the learning curve identifies as most data-starved.

Ensemble detection combining rule-based and LLM layers. The per-detector analysis reveals a complementary relationship: GPT-4o dominates on lifecycle-ownership detectors (FRAGMENT VIEW FIELD RETENTION, STATE HOLDER LEAK), while fine-tuned StarCoder2 is the only model with non-zero recall on THREADED UI REFERENCE. The planned work will design and evaluate an ensemble that routes code snippets to the rule-based LEAKSCOPE detector, the fine-tuned LLM, or both, and resolves disagreements through a confidence-weighted voting scheme. The ensemble will be evaluated on the full held-out split and on a newly collected Kotlin-first extension of the corpus.

Kotlin corpus expansion and cross-language generalization. The current training corpus is predominantly Java-based (94.7% of the 627-project corpus). Modern Android development increasingly uses Kotlin, with Kotlin coroutines, flows, and Jetpack Compose introducing new lifecycle-retention patterns [35]. The planned work will extend the labeled corpus to Kotlin-first projects, re-run fine-tuning, and assess whether the StarCoder2-7B adapter generalizes across

languages or requires language-specific training data.

5.5 Research Timeline

Table 5.5: Planned research timeline from current state through dissertation defense.

Activity	Target period
Finalize manual validation; complete RQ3/RQ4 results	Spring 2026
Submit LLM fine-tuning study to ICSE 2027	Summer 2026
Multi-seed learning curves; data collection for rare detectors	Summer 2026
Ensemble rule-based + LLM detection study	Fall 2026
Developer study: practitioner assessment of LEAKSCOPE and LLM reports	Fall 2026
Dissertation writing and defense	2026

BIBLIOGRAPHY

[1] Android Developers, "Fragment lifecycle," [Online]. Available: https://developer.android.com/guide/fragments/lifecycle, accessed: Apr. 26, 2026.

[2] ——, "View binding," [Online]. Available: https://developer.android.com/topic/libraries/view-binding, accessed: Apr. 26, 2026.

[3] ——, "Better performance through threading," [Online]. Available: https://developer.android.com/topic/performance/threads, accessed: Apr. 26, 2026.

[4] ——, "ViewModel overview," [Online]. Available: https://developer.android.com/topic/libraries/architecture/viewmodel, accessed: Apr. 26, 2026.

[5] ——, "Manage your app's memory," [Online]. Available: https://developer.android.com/topic/performance/memory, accessed: Apr. 26, 2026.

[6] ——, "Provide custom back navigation," [Online]. Available: https://developer.android.com/guide/navigation/navigation-custom-back, accessed: Apr. 26, 2026.

[7] ——, "BillingClient," [Online]. Available: https://developer.android.com/reference/com/android/billingclient/api/BillingClient, accessed: Apr. 26, 2026.

[8] ——, "Services overview," [Online]. Available: https://developer.android.com/develop/background-work/services, accessed: Apr. 26, 2026.

[9] ——, "Improve your code with lint checks," [Online]. Available: https://developer.android.com/studio/write/lint, accessed: Apr. 26, 2026.

[10] SpotBugs Project, "SpotBugs: Find bugs in java programs," [Online]. Available: https://spotbugs.github.io/, accessed: Apr. 26, 2026.

[11] Square, "LeakCanary: A memory leak detection library for android," [Online]. Available: https://square.github.io/leakcanary/, accessed: Apr. 26, 2026.

[12] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. L. Traon, D. Octeau, and P. McDaniel, "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '14. Association for Computing Machinery, 2014, pp. 259–269.

[13] R. Vallée-Rai, P. Co, E. Gagnon, L. J. Hendren, P. Lam, and V. Sundaresan, "Soot: A java bytecode optimization framework," in *Proceedings of the 1999 Conference of the Centre for Advanced Studies on Collaborative Research*, ser. CASCON '99. Mississauga, ON, Canada: IBM Press, 1999, p. 13.

[14] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[15] Android Developers, “Fragments,” [Online]. Available: https://developer.android.com/guide/fragments, accessed: Apr. 26, 2026.

[16] ——, “State holders and ui state,” [Online]. Available: https://developer.android.com/topic/architecture/ui-layer/stateholders, accessed: Apr. 26, 2026.

[17] ——, “WindowInfoTracker,” [Online]. Available: https://developer.android.com/reference/androidx/window/layout/WindowInfoTracker, accessed: Apr. 26, 2026.

[18] ——, “Service,” [Online]. Available: https://developer.android.com/reference/android/app/Service, accessed: Apr. 26, 2026.

[19] N. Mitchell and G. Sevitsky, “Leakbot: An automated and lightweight tool for diagnosing memory leaks in large java applications,” in *Proceedings of the 17th European Conference on Object-Oriented Programming (ECOOP)*. Springer, 2003, pp. 351–377.

[20] C. Guo, J. Zhang, J. Yan, Z. Zhang, and Y. Zhang, “Characterizing and detecting resource leaks in android applications,” in *Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2013, pp. 389–398.

[21] T. Wu, J. Liu, X. Deng, J. Yan, and J. Zhang, “Relda2: an effective static analysis tool for resource leak detection in android apps,” in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2016, pp. 762–767.

[22] Android Developers, “Data binding library,” [Online]. Available: https://developer.android.com/topic/libraries/data-binding, accessed: Apr. 26, 2026.

[23] DP-Hridayan, “aShellYou: An android shell application,” [Online]. Available: https://github.com/DP-Hridayan/aShellYou, accessed: Apr. 26, 2026.

[24] M. Sulír and J. Porubän, “A quantitative study of java software buildability,” in *Proceedings of the 7th International Workshop on Evaluation and Usability of Programming Languages and Tools*, ser. PLATEAU 2016. New York, NY, USA: Association for Computing Machinery, 2016, pp. 17–25. [Online]. Available: https://doi.org/10.1145/3001878.3001882

[25] F. Wei, S. Roy, X. Ou, and Robby, “Amandroid: A precise and general inter-component data flow analysis framework for security vetting of android apps,” *ACM Transactions on Privacy and Security*, vol. 21, no. 3, pp. 1–32, 2018.

[26] L. Li, A. Bartel, T. F. Bissyandé, J. Klein, Y. L. Traon, S. Arzt, S. Rasthofer, E. Bodden, D. Octeau, and P. McDaniel, “IccTA: Detecting inter-component privacy leaks in android apps,” in *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering*, ser. ICSE ’15. IEEE Computer Society, 2015, pp. 280–291.

[27] M. I. Gordon, D. Kim, J. Perkins, L. Gilham, N. Nguyen, and M. Rinard, “Information-flow analysis of android applications in droidsafe,” in *Proceedings of the 22nd Annual Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2015.

[28] L. Li, T. F. Bissyand’e, M. Papadakis, S. Rasthofer, A. Bartel, D. Octeau, J. Klein, and Y. Le Traon, “Static analysis of android apps: A systematic literature review,” *Information and Software Technology*, vol. 88, pp. 67–95, 2017.

[29] T. Su, L. Fan, S. Chen, Y. Liu, L. Xu, G. Pu, and Z. Su, "Why my app crashes? understanding and benchmarking framework-specific exceptions of android apps," *IEEE Transactions on Software Engineering*, vol. 48, no. 4, pp. 1115–1137, 2022.

[30] D. Wu, F. Mu, L. Shi, Z. Guo, K. Liu, W. Zhuang, Y. Zhong, and L. Zhang, "iSMELL: Assembling LLMs with expert toolsets for code smell detection and refactoring," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. Association for Computing Machinery, 2024, pp. 1345–1357.

[31] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat *et al.*, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[32] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier *et al.*, "StarCoder 2 and the stack v2: The next generation," *arXiv preprint arXiv:2402.19173*, 2024.

[33] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proceedings of the 10th International Conference on Learning Representations*, ser. ICLR '22, 2022.

[34] S. Strande, H. Cai, M. Tatineni, W. Pfeiffer, C. Irving, A. Majumdar, D. Mishin, R. S. Sinkovits, M. Norman, N. Wolter, T. Cooper, I. Altintas, M. Kandes, I. Perez, M. Shantharam, M. Thomas, S. Sivagnanam, and T. Hutton, "Expanse: Computing without boundaries: Architecture, deployment, and early operations experiences of a supercomputer designed for the rapid evolution in science and engineering," in *Practice and Experience in Advanced Research Computing 2021: Evolution Across All Dimensions*, ser. PEARC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: https://doi.org/10.1145/3437359.3465588

[35] B. Mateus and M. Martinez, "On the adoption, usage and evolution of kotlin features in android development," in *Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2020, pp. 1–11.